

P1-Unix Shell

(CSS 430 B, Winter 2025)

You can use the starter code at <https://github.com/pisan430/p1-shell-starter>.

Project 1—UNIX Shell

This project consists of designing a C program to serve as a shell interface that accepts user commands and then executes each command in a separate process. Your implementation will support input and output redirection, as well as pipes as a form of IPC between a pair of commands. Completing this project will involve using the UNIX `fork()`, `exec()`, `wait()`, `dup2()`, and `pipe()` system calls and can be completed on any Linux, UNIX, or macOS system.

Overview

A shell interface gives the user a prompt, after which the next command is entered. The example below illustrates the prompt `osh>` and the user's next command: `cat prog.c`. (This command displays the file `prog.c` on the terminal using the UNIX `cat` command.)

```
osh> cat prog.c
```

One technique for implementing a shell interface is to have the parent process first read what the user enters on the command line (in this case, `cat prog.c`) and then create a separate child process that performs the command. Unless otherwise specified, the parent process waits for the child to exit before continuing. This is similar in functionality to the new process creation illustrated in Figure 3.9.

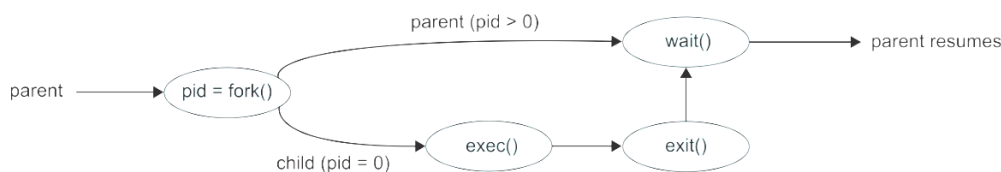


Figure 3.9 Process creation using the `fork()` system call.

However, UNIX shells typically also allow the child process to run in the background, or concurrently. To accomplish this, we add an ampersand (&) at the end of the command. Thus, if we rewrite the above command as

```
osh>cat prog.c &
```

the parent and child processes will run concurrently.

The separate child process is created using the `fork()` system call, and the user's command is executed using one of the system calls in the `exec()` family (as described in Section 3.3.1).

A C program that provides the general operations of a command-line shell is supplied in Figure 3.36. The main() function presents the prompt osh-> and outlines the steps to be taken after input from the user has been read. The main() function continually loops as long as should_run equals 1; when the user enters exit at the prompt, your program will set should_run to 0 and terminate.

```
#include <stdio.h>
#include <unistd.h>

#define MAX_LINE 80 /* The maximum length command */

int main(void)
{
    char *args[MAX_LINE/2 + 1]; /* command line arguments */
    int should_run = 1; /* flag to determine when to exit program */

    while (should_run) {
        printf("osh> ");
        fflush(stdout);

        /**
         * After reading user input, the steps are:
         * (1) fork a child process using fork()
         * (2) the child process will invoke execvp()
         * (3) parent will invoke wait() unless command included &
         */
        (2.5) (4) Parent and child call close(fd)
    }
    return 0;
}
```

EXECVP EXAMPLE
(<https://stackoverflow.com/a/27542113/27961837>):

```
char *cmd = "ls";
char *argv[3];
argv[0] = "ls";
argv[1] = "-la";
argv[2] = NULL;

// Run run "ls -la" as if it were a command
execvp(cmd, argv);
```

execvp(const char* file, char* const argv[])

System call used to replace current process image (including memory, code and stack) with a new process image. The PID remains the same.

file: Name of file (program) to execute. Unlike exec(), execvp can search PATH (so you don't have to specify full path)

argv[]: Array of (null-terminated) strings representing arguments to pass to the program. argv[0] is the program name.

On failure, execvp returns -1. Otherwise, it does not return because the current process is replaced by the new program.

If the command included pipes:

(1) The parent will need to create a pipe for each connection between commands. Pipe must be created before forking so it's visible to parent & child.

Each pipe() call returns two file descriptors: pipe[i][0] for reading and pipe[i][1] for writing.

(2) The parent will call fork after each pipe (for each command).

(3) The children need to redirect stdin to the pipe read-end and stdout to pipe write-end, using dup2 system call

dup2(oldfd, newfd) duplicates oldfd to newfd (if newfd is already open, it's closed first).

Duplicates read end of the pipe(pipeFD[READ]) to the file descriptor 0;
> fd=0 is the standard input (stdin).
> fd=1 is used by cout
> fd=2 is used by cerr

so read operations after dup2(pipeFD[0], STDIN_FILENO) will read from the pipe.

Figure 3.36 Outline of simple shell.

```
#include <stdio.h>
#include <unistd.h>

#define MAX_LINE 80 /* The maximum length command */

int main(void)
{
    char *args[MAX_LINE/2 + 1]; /* command line arguments */
    int should_run = 1; /* flag to determine when to exit program */

    while (should_run) {
        printf("osh>");
        fflush(stdout);

        /**
         * After reading user input, the steps are:
         * (1) fork a child process using fork()
         * (2) the child process will invoke execvp()
         * (3) parent will invoke wait() unless command included &
         */
    }

    return 0;
}
```

Figure 3.36 Outline of simple shell.

This project is organized into several parts:

- **1.** Creating the child process and executing the command in the child
- **2.** Providing a history feature
- **3.** Adding support of input and output redirection
- **4.** Allowing the parent and child processes to communicate via a pipe

Executing Command in a Child Process

The first task is to modify the `main()` function in Figure 3.32 so that a child process is forked and executes the command specified by the user. This will require parsing what the user has entered into separate tokens and storing the tokens in an array of character strings (`args` in Figure 3.32).

Hint: You might find the `tokenize` example in <https://github.com/pisan430/c-for-os> helpful.

For example, if the user enters the command `ps -ael` at the `osh>` prompt, the values stored in the `args` array are:

```
args[0] = "ps"  
args[1] = "-ael"  
args[2] = NULL
```

This `args` array will be passed to the `execvp()` function, which has the following prototype:

```
execvp(char *command, char *params[])
```

Here, `command` represents the command to be performed and `params` stores the parameters to this command. For this project, the `execvp()` function should be invoked as `execvp(args[0], args)`. Be sure to check whether the user included `&` to determine whether or not the parent process is to wait for the child to exit.

Creating a History Feature

The next task is to modify the shell interface program so that it provides a **history** feature to allow a user to execute the most recent command by entering `!!`. For example, if a user enters the command `ls -l`, she can then execute that command again by entering `!!` at the prompt. Any command executed in this fashion should be echoed on the user's screen, and the command should also be placed in the history buffer as the next command.

Your program should also manage basic error handling. If there is no recent command in the history, entering `!!` should result in a message “No commands in history.”

Redirecting Input and Output

Your shell should then be modified to support the ‘>’ and ‘<’ redirection operators, where ‘>’ redirects the output of a command to a file and ‘<’ redirects the input to a command from a file. For example, if a user enters

```
osh> ls > out.txt
```

the output from the `ls` command will be redirected to the file `out.txt`. Similarly, input can be redirected as well. For example, if the user enters

```
osh> sort < in.txt
```

the file `in.txt` will serve as input to the `sort` command.

Managing the redirection of both input and output will involve using the `dup2()` function, which duplicates an existing file descriptor to another file descriptor. For example, if `fd` is a file descriptor to the file `out.txt`, the call

```
dup2(fd, STDOUT_FILENO);
```

duplicates `fd` to standard output (the terminal). This means that any writes to standard output will in fact be sent to the `out.txt` file.

You can assume that commands will contain either one input or one output redirection and will not contain both. In other words, you do not have to be concerned with command sequences such as `sort < in.txt > out.txt`.

Communication via a Pipe

The final modification to your shell is to allow the output of one command to serve as input to another using a pipe. For example, the following command sequence

```
osh> ls -l | less
```

has the output of the command `ls -l` serve as the input to the `less` command. Both the `ls` and `less` commands will run as separate processes and will communicate using the UNIX `pipe()` function described in Section 3.7.4 [Links to an external site.](#). Perhaps the easiest way to create these separate processes is to have the parent process create the child process (which will execute `ls -l`). This child will also create another child process (which will execute `less`) and will establish a pipe between itself and the child process it creates. Implementing pipe functionality will also require using the `dup2()` function as described in the previous section. Finally, although several commands can be chained together using multiple pipes, you can assume that commands will contain only one pipe character and will not be combined with any redirection operators.

Hints

- If a command ends with ";" then the parent must wait() for that child to finish
- If the command ends with "&", the parent does not wait() for the child to finish
- If the last command on the line does not end with ";" or "&", then treat it as if the user had typed ";" (ie, parent waits)
- "ls ; sleep 5; ls -al"
 - Execute ls, wait for it to finish
 - Execute sleep 5; wait for it to finish
 - Execute ls -al
- "ls ; sleep 5 & ls -al"
 - Execute ls, wait for it to finish
 - Execute sleep 5; do NOT wait for it to finish
 - Execute ls -al
- The following command might help with understanding parent-child relationships

```
csslab1:~$ ps auxf | cat | tac | cat | tac | grep pisan
root      13359  0.0  0.0 167424  6088 ?        Ss   21:35   0:00 \_ sshd: pisa [priv]
pisan     13361  0.0  0.0 167424  2380 ?        S    21:35   0:00 \_ sshd: pisan@pts/4
pisan     13362  0.3  0.0 124636  2880 pts/4    Ss   21:35   0:00 \_ -bash
pisan     13394  0.0  0.0 164032  2076 pts/4    R+   21:35   0:00 \_ ps auxf
pisan     13395  0.0  0.0 108076   356 pts/4    S+   21:35   0:00 \_ cat
pisan     13396  0.0  0.0 124636  1300 pts/4    D+   21:35   0:00 \_ -bash
pisan     13397  0.0  0.0 108076   356 pts/4    S+   21:35   0:00 \_ cat
pisan     13398  0.0  0.0 124636  1300 pts/4    D+   21:35   0:00 \_ -bash
pisan     13399  0.0  0.0 112816   980 pts/4    S+   21:35   0:00 \_ grep --color=auto pisan
```

Grading Rubric

Here are the tests we will use for grading: total of 25 points

- ls
- ls -al
- ls & whoami ;

5
2
3

So we will run your Shell on Linux, and try each of the 7 commands opposite. If it works for "ls", you get 5 points. If it works for "ls -al" you get another 2 points. And so on.

- !!

3

- ls > junk.txt
- cat < junk.txt

5
2

Advice: this project is quite challenging - part 4, on piping, in particular. So start early, and ask questions to clarify what's required

- ls | wc

5

$\Sigma = 25$

Bonus: +3 if your program can handle. Note the backticks around whoami. The result of that command is used as the argument for grep.

```
$ ps auxf | cat | tac | cat | tac | grep `whoami`
```

Bonus: +1 Easter egg. Your program should output some ASCII art when the command 'ascii' is entered.

```
$ ascii
```

```
| \_ / |  
/ @ @ \  
( > o < )  
>>x<<  
/ o \  

```

```
***** ( \_ / )  
* "Purrrfectly pleasant" * (= ' .' =)  
* Poppy Prinz * (" ) _ ( "  
* (pprinz@example.com) *  
*****
```